

Placement Empowerment Program
Cloud Computing and DevOps Centre

Write a Shell Script to Monitor Logs

Create a script that monitors server logs for errors and alerts you.

Name: Neithal Pillai

Department : IT

Introduction and Overview

In this **Proof of Concept (PoC)**, we will learn how to monitor log files in real time using a **Shell Script** on a **Windows laptop with Git Bash**. This process involves creating a log monitoring script, setting up continuous log tracking, and triggering alerts when errors are detected. By following these steps, you'll gain hands-on experience with **log file analysis and automation**, which is a crucial skill for system administration and debugging.

Objective

The goal of this project is to:

1. **Set up a log monitoring script** using a Shell Script.
2. **Continuously track log files** for errors in real time.
3. **Trigger alerts** when specific error patterns are detected in the logs.

Importance of Log Monitoring

Effective log monitoring is essential for system administrators and developers, as it allows for early detection of issues and proactive troubleshooting. It offers several advantages, such as:

- **Proactive Issue Detection:** Identifies errors in real-time, reducing downtime.
- **Automated Alerting:** Notifies users when critical errors occur.
- **Enhanced Debugging:** Helps diagnose issues efficiently by analyzing logs.
- **System Optimization:** Provides insights into system performance and stability.

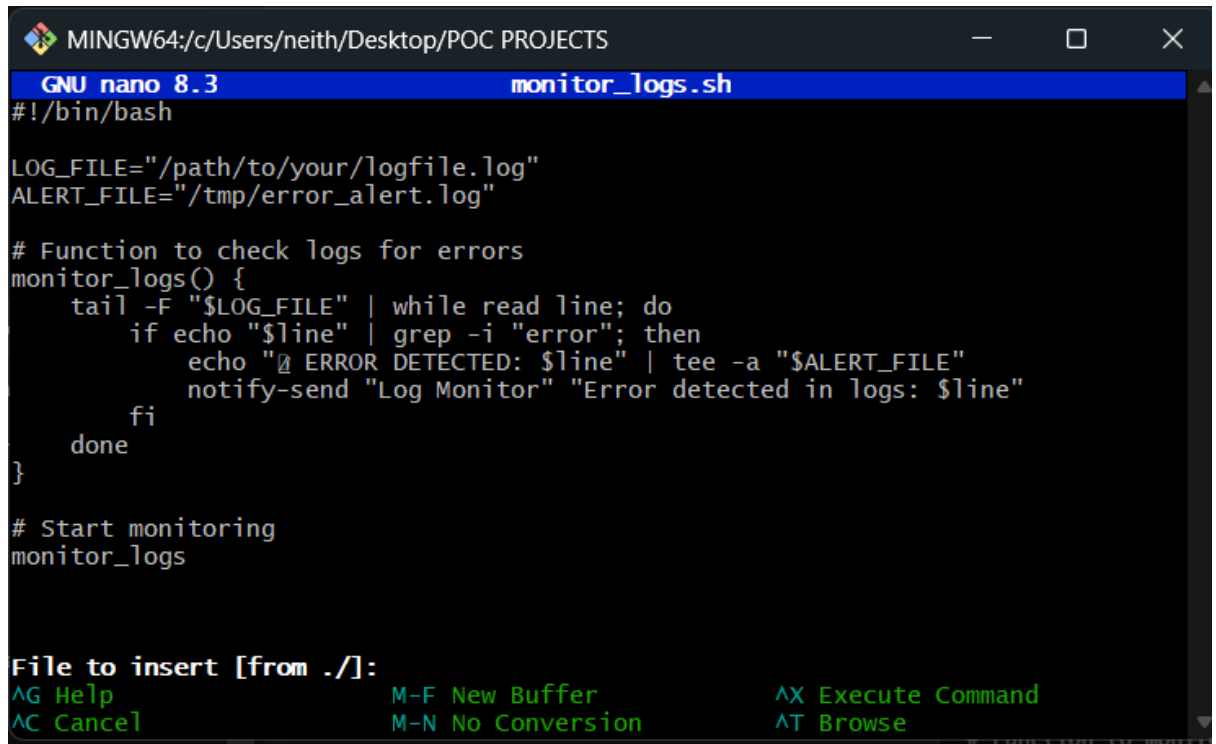
Steps For Execution:

Step 1: Open Git Bash

- Launched **Git Bash** on a Windows laptop.
- Navigated to the **POC PROJECTS** folder where the log file (mylog.log) is stored.

Step 2: Open the Shell Script in Nano

- Used nano to open the monitor_logs.sh script for editing.
- The script was being modified to **monitor log files for errors**.



```
MINGW64:/c/Users/neith/Desktop/POC PROJECTS
GNU nano 8.3 monitor_logs.sh
#!/bin/bash

LOG_FILE="/path/to/your/logfile.log"
ALERT_FILE="/tmp/error_alert.log"

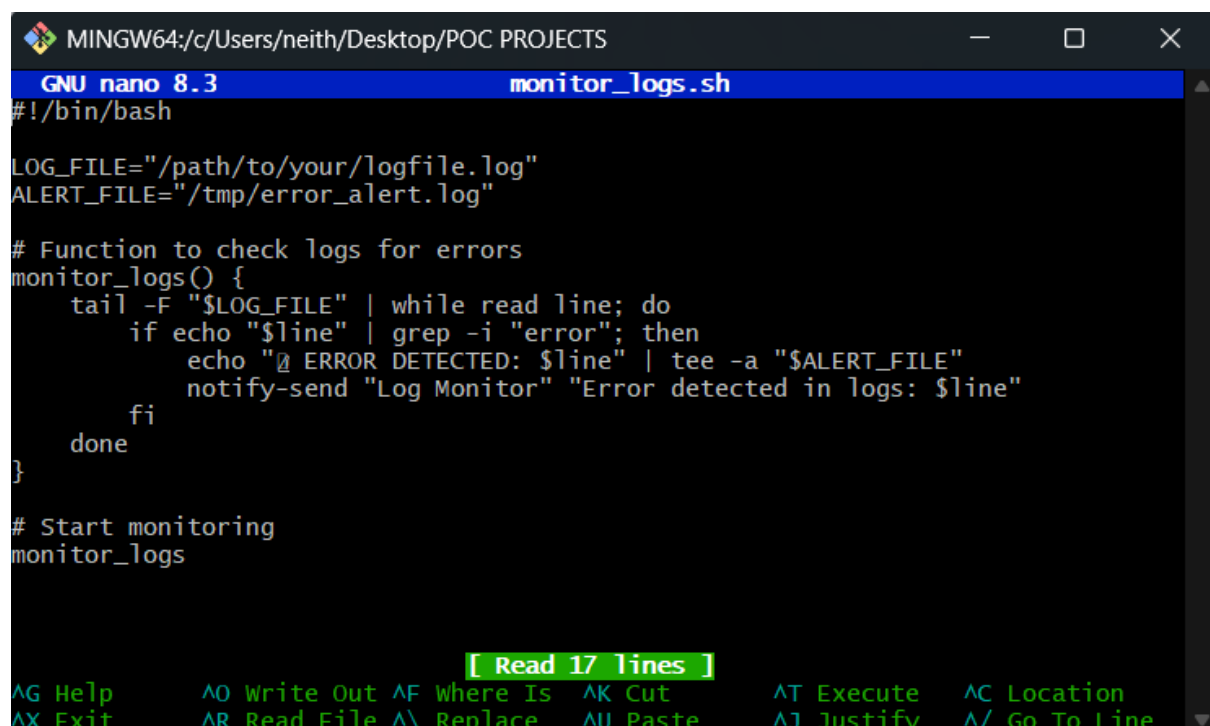
# Function to check logs for errors
monitor_logs() {
    tail -F "$LOG_FILE" | while read line; do
        if echo "$line" | grep -i "error"; then
            echo "@ ERROR DETECTED: $line" | tee -a "$ALERT_FILE"
            notify-send "Log Monitor" "Error detected in logs: $line"
        fi
    done
}

# Start monitoring
monitor_logs

File to insert [from ./]:
^G Help          M-F New Buffer    ^X Execute Command
^C Cancel        M-N No Conversion ^T Browse
```

Step 3: Save and Exit the Script

- Pressed **Ctrl + X** to exit the nano editor.



```
MINGW64:/c/Users/neith/Desktop/POC PROJECTS
GNU nano 8.3 monitor_logs.sh
#!/bin/bash

LOG_FILE="/path/to/your/logfile.log"
ALERT_FILE="/tmp/error_alert.log"

# Function to check logs for errors
monitor_logs() {
    tail -F "$LOG_FILE" | while read line; do
        if echo "$line" | grep -i "error"; then
            echo "@ ERROR DETECTED: $line" | tee -a "$ALERT_FILE"
            notify-send "Log Monitor" "Error detected in logs: $line"
        fi
    done
}

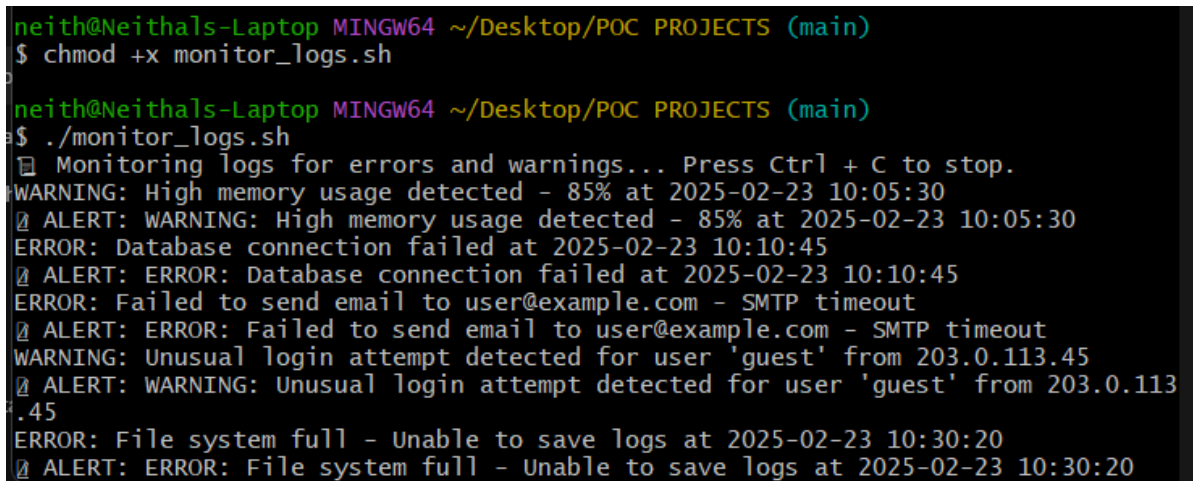
# Start monitoring
monitor_logs

[ Read 17 lines ]
^G Help          ^O Write Out      ^F Where Is      ^K Cut           ^T Execute       ^C Location
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^_ Go To Line
```

- **Pressed Y** to confirm saving the script.
- **Pressed Enter** to finalize and return to Git Bash.

Step 4: Verify the Script Execution

- Checked that the script was correctly modified.
- Executed the script using Git Bash to ensure it **monitors the log file** for errors.



```
neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ chmod +x monitor_logs.sh

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ ./monitor_logs.sh
[?] Monitoring logs for errors and warnings... Press Ctrl + C to stop.
WARNING: High memory usage detected - 85% at 2025-02-23 10:05:30
[?] ALERT: WARNING: High memory usage detected - 85% at 2025-02-23 10:05:30
ERROR: Database connection failed at 2025-02-23 10:10:45
[?] ALERT: ERROR: Database connection failed at 2025-02-23 10:10:45
ERROR: Failed to send email to user@example.com - SMTP timeout
[?] ALERT: ERROR: Failed to send email to user@example.com - SMTP timeout
WARNING: Unusual login attempt detected for user 'guest' from 203.0.113.45
[?] ALERT: WARNING: Unusual login attempt detected for user 'guest' from 203.0.113.45
ERROR: File system full - Unable to save logs at 2025-02-23 10:30:20
[?] ALERT: ERROR: File system full - Unable to save logs at 2025-02-23 10:30:20
```

Step 5: Conclusion

- The log monitoring script was successfully updated and executed.
- The process is now streamlined for continuous log monitoring.

What I Learned from This Process:

- **Log Monitoring:** Tracking and filtering errors in real-time.
- **Shell Scripting:** Writing and executing scripts in Git Bash.
- **Automation:** Detecting and logging errors efficiently.
- **System Administration:** Using logs for debugging and troubleshooting